



Introduction to C Programming

Lecture Notes

Mithun A. V.

Contents

I	Programming Fundamentals	
1	Introduction to Programming	9
1.1	Program Development Cycle	9
1.2	Algorithm	10
1.3	Flowchart	10
1.3.1	Flowchart Symbols	10
1.3.2	Guidelines for preparing flowcharts	10
1.3.3	Benefits of flowcharts	11
1.3.4	Limitations of flowcharts	11
1.4	Some Examples	11
1.5	Try it yourself	13
II	Programming with C	
2	Basic Elements of C	17
2.1	History of C	17
2.2	Importance of C	17
2.3	Basic Structure of C Programs	18
2.4	Executing a C Program	18
2.5	Source Code, Object Code & Executable File	19
2.6	File Extension	20
2.7	Character Set	20

2.8	C Tokens	20
2.9	Keywords	21
2.10	Identifiers	21
2.11	Constants	21
2.11.1	Integer Constants	21
2.11.2	Real Constants	22
2.11.3	Single Character Constants	22
2.11.4	String Constants	22
2.11.5	Backslash Character Constants	22
2.12	Variables	23
2.13	Data Types	23
2.13.1	Primary(Fundamental) Data Types	23
2.14	Declaration of Variables	24
2.14.1	Primary Type Declaration	24
2.14.2	User-Defined Type Declaration	25
2.15	Defining Symbolic Constants	25
2.16	Operators in C	26
2.16.1	Arithmetic Operators	26
2.16.2	Relational Operators	27
2.16.3	Logical Operators	28
2.16.4	Assignment Operators	28
2.16.5	Increment (++) and Decrement (--) Operators	29
2.16.6	Conditional Operator	29
2.16.7	Bitwise Operators	30
2.16.8	Special Operators	31
2.17	Type Conversion in Expressions	32
2.17.1	Implicit Type Conversion	32
2.17.2	Explicit Type Conversion	32
2.17.3	Operator Precedence and Associativity	32
3	Program Control Statements	35
3.1	Managing Input and Output	35
3.1.1	Reading a Character	35
3.1.2	Writing a Character	35
3.1.3	Formatted Input	36
3.1.4	Formatted Output	37
3.2	Decision Making and Branching	38
3.2.1	if Statement	38
3.2.2	The switch Statement	44
3.2.3	The goto Statement	45
3.3	Looping	46
3.3.1	The while statement	47
3.3.2	The do - while Statement	47
3.3.3	The for Statement	48
3.3.4	break Statement	49
3.3.5	continue Statement	49

4	Arrays & Strings	51
4.1	Introduction	51
4.2	One Dimensional Arrays	51
4.2.1	Declaration of One Dimensional Arrays	52
4.2.2	Initialization of One Dimensional Arrays	52
4.3	Two Dimensional Arrays	53
4.3.1	Initialization of Two Dimensional Arrays	53
4.4	Multidimensional Arrays	53
4.5	Strings	54
4.5.1	Declaring and Initializing String Variables	54
4.5.2	Reading Strings From Terminal	55
4.5.3	Writing Strings to Function	56
4.5.4	Arithmetic Operations on Characters	56
4.5.5	String Handling Functions	56
4.5.6	Table of Strings	58
	Bibliography	59
	Books	59



Programming Fundamentals

1	Introduction to Programming	9
1.1	Program Development Cycle	
1.2	Algorithm	
1.3	Flowchart	
1.4	Some Examples	
1.5	Try it yourself	

1. Introduction to Programming

A **computer program** consists of a series of instructions that a computer processes to perform the required operation. Instructions in the computer programs are written using **programming languages**. A computer program consists of three basic parts:

1. The instructions to be executed
2. The order in which the instructions are to be executed
3. The data required to perform the instructions (input)

1.1 Program Development Cycle

Program development cycle has the following phases:

1. **Analyze/Define the problem:** The programmer has to analyze the problem thoroughly to know about the scope of the problem.
2. **Task analysis:** Next, the programmer has to develop various solutions to solve the problem. From these solutions, the optimum solution is chosen to solve the problem comfortably.
3. **Develop algorithm:** After selecting the optimum solution, algorithm is developed to depict the logic of the selected solution.
4. **Test algorithm for accuracy:** Algorithm has to be checked for accuracy before it is converted to actual code. This will enable the programmer to identify major logical errors at an earlier stage of program development.
5. **Coding:** After testing the algorithm, actual code is prepared in a preferred programming language.
6. **Test and debug the program:** The compiler of the programming languages test the program for syntax errors. Also, the results obtained from the program is compared with the results calculated manually using a number of sample input data.
7. **Documentation:** After all the errors in the program are corrected, the programmer has to support the program with sufficient document, so that the users can understand how to operate the program.
8. **Implementation:** In this phase, the program is installed on the end user's machine. This phase can be considered as the final testing, because only after using the program, the user

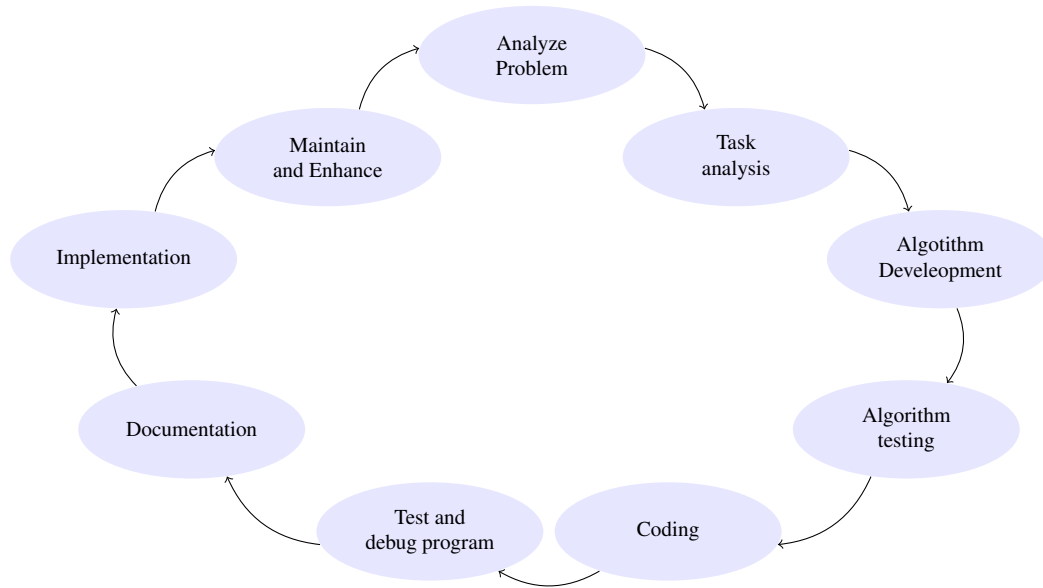


Figure 1.1: Program Development Cycle

can point out the drawbacks, if any.

9. **Maintenance and Enhancement:** After the program is implemented, it should be maintained by taking care of the changing requirements of its users. The program should be regularly enhanced by adding additional features.

1.2 Algorithm

An algorithm is defined as a finite sequence of explicit instructions that, when provided with a set of input values, produces an output and then terminates. An algorithm should have the following properties:

- There must be no ambiguity in any instruction
- There should not be any uncertainty about which instruction should be executed next
- The execution of the algorithm should conclude after a finite number of steps
- Algorithms must be general enough to deal with any circumstances

Note: Refer some examples discussed in the classroom

1.3 Flowchart

A **flowchart** is a pictorial representation of an algorithm in which the steps are drawn in the form of different shaped boxes and the logical flows are indicated by interconnecting arrows. The boxes represent operation and the arrows represent the sequence in which operations are implemented.

1.3.1 Flowchart Symbols

Different symbols are used for creating flow chart are given in Table 1.1.

1.3.2 Guidelines for preparing flowcharts

- The flowchart should be clear, neat and easy to follow
- The flowchart must have logical start and finish
- Only one flow line should come out of a process symbol

Symbol	Symbol Name	Description
	Flow lines	Indicate the sequence of steps and direction of flow.
	Terminal	Represents the beginning and end of a flowchart.
	Processing	Represents arithmetic or data movement instructions
	Input/Output	Represents information leaving or entering the system
	Decision	Represents a decision. Program flows in one of the routes.
	Connector	Used to join different flow lines
	Annotation	Used to give additional information about a symbol.

Table 1.1: Flowchart Symbols

- Only one flow line should enter a decision symbol However two or three flow lines leave the decision symbol
- In case of complex flowcharts, connector symbols are used to reduce the number of flow lines
- Intersection of flow lines should be avoided

1.3.3 Benefits of flowcharts

- **Makes logic clear:** Since flowcharts provide a pictorial representation of the task, it makes the logic easier to follow.
- **Communication:** Flowchart is a better way of communicating the logic of a system to all concerned.
- **Effective in Coding:** The programmers can plan the coding effectively as they know where to begin and where to end
- **Appropriate Documentation:** Flowcharts provide as a good documentation tool

1.3.4 Limitations of flowcharts

- **Complex:** If the program is very large, the flowchart may continue for many pages making it difficult to follow.
- **Costly:** Time required for creating flowchart for a complex program is too much
- **Difficult to modify:** Any modification to a flowchart usually requires redrawing the entire logic again
- **No update:** Usually programs are updated regularly. However the corresponding update on the flowchart may not take place

1.4 Some Examples

Example 1.1. Write an algorithm to find the roots of a quadratic equation. Draw the flowchart for the same.

Step 1 Start

Step 2 Get the coefficients of x^2 , x and the constant (a , b and c respectively)

Step 3 Find the discriminant $d = b^2 - 4ac$

Step 4 If $d = 0$ go to **Step 6**, else go to **Step 5**.

Step 5 If $d > 0$ go to **Step 7**, else go to **Step 8**.

Step 6 Print "Roots are real and equal" and it is $\frac{-b}{2a}$. Go to **Step 9**.

Step 7 Print "Roots real and distinct" and they are is $\frac{-b+\sqrt{d}}{2a}$ and $\frac{-b-\sqrt{d}}{2a}$. Go to **Step 9**.

Step 8 Print "Roots are imaginary" and they are is $\frac{-b+\sqrt{-d}i}{2a}$ and $\frac{-b-\sqrt{-d}i}{2a}$. Go to **Step 9**.

Step 9 Stop.

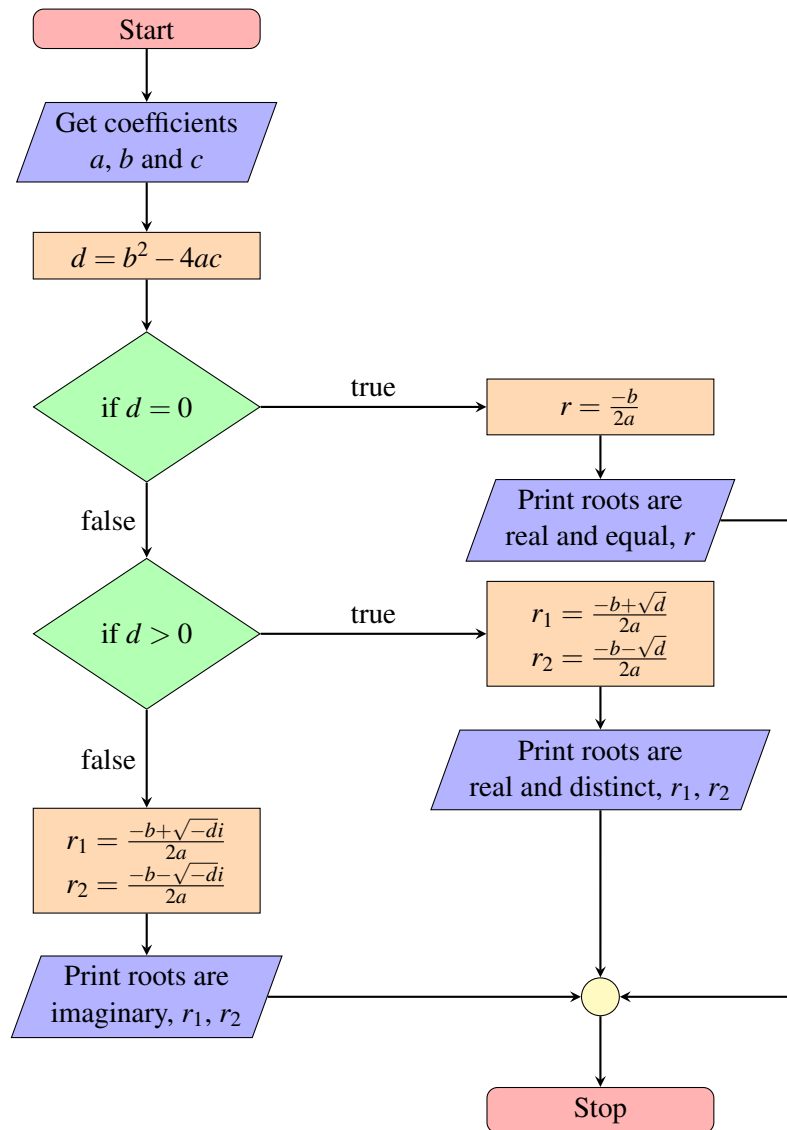


Figure 1.2: Flowchart for Example 1.1

Example 1.2. Write an algorithm to generate n numbers in the fibonacci sequence. Draw the flowchart for the same.

Step 1 Start

Step 2 Get n , the number of terms to be generated.

Step 3 Let $f_1 = 0$, $f_2 = 1$, $count = 1$

Step 4 If $count \leq n$, go to **Step 5**, else go to **Step 9**.

Step 5 Print f_1

Step 6 $f_3 = f_1 + f_2$

Step 7 $f_1 = f_2$, $f_2 = f_3$

Step 8 $count = count + 1$. Go to **Step 4**.

Step 9 Stop.

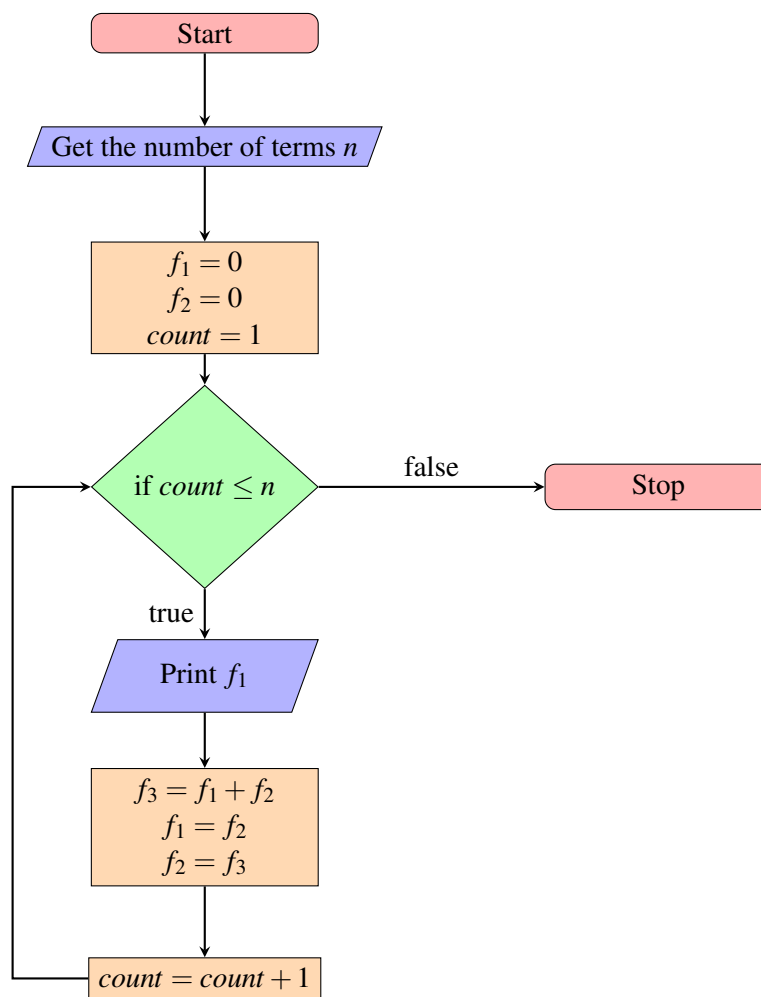


Figure 1.3: Flowchart for Example 1.1

1.5 Try it yourself

Exercise 1.1 Write an algorithm and draw the flowchart to check whether a number is odd or even. ■

Exercise 1.2 Write an algorithm and draw the flowchart to print the multiples of 7 between 100 and 200. ■

Exercise 1.3 Write an algorithm and draw the flowchart to check whether a number is prime or not. ■

Exercise 1.4 Write an algorithm and draw the flowchart to find the largest among three given numbers. ■

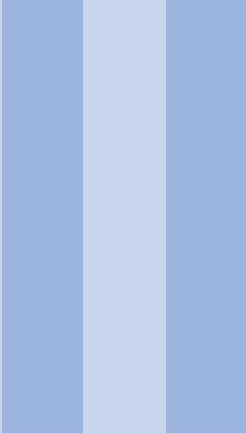
Exercise 1.5 Write an algorithm and draw the flowchart to find the sum of the digits of a number. ■

Exercise 1.6 Write an algorithm and draw the flowchart to check whether a number is odd or even. ■

Exercise 1.7 Write an algorithm and draw the flowchart to check whether a number is perfect or not. ■

Exercise 1.8 Write an algorithm and draw the flowchart to check whether a number is Armstrong number. ■

Exercise 1.9 Write an algorithm and draw the flowchart to generate first n prime numbers. ■



Programming with C

2	Basic Elements of C	17
2.1	History of C	
2.2	Importance of C	
2.3	Basic Structure of C Programs	
2.4	Executing a C Program	
2.5	Source Code, Object Code & Executable File	
2.6	File Extension	
2.7	Character Set	
2.8	C Tokens	
2.9	Keywords	
2.10	Identifiers	
2.11	Constants	
2.12	Variables	
2.13	Data Types	
2.14	Declaration of Variables	
2.15	Defining Symbolic Constants	
2.16	Operators in C	
2.17	Type Conversion in Expressions	
3	Program Control Statements	35
3.1	Managing Input and Output	
3.2	Decision Making and Branching	
3.3	Looping	
4	Arrays & Strings	51
4.1	Introduction	
4.2	One Dimensional Arrays	
4.3	Two Dimensional Arrays	
4.4	Multidimensional Arrays	
4.5	Strings	
	Bibliography	59
	Books	

2. Basic Elements of C

2.1 History of C

C is a structured high level machine independent language.

Year	Language	By	Description
1960	ALGOL (ALGOrithmic Language)	International Group	Root of all programming languages
1967	BCPL (Basic Combined Programming Language)	Martin Richards	Typeless Language
1970	B	Ken Thompson	Typeless Language
1972	Traditional C	Dennis Ritchie	Added the concept of data types
1978	K&R C	Kernighan and Ritchie	Published a book
1989	ANSI C	ANSI (American National Standards Institute)	ANSI formed a committee and approved a version of C
1990	ANSI/ISO C	ISO (International Standards Organization)	Approved by ISO
1999	C99	Standardization Committee	Added features from Java and C++

Table 2.1: History of C

2.2 Importance of C

- C is a robust language with lot of built-in functions and operators. Hence it can be used to write complex programs.
- C has capabilities of assembly language and high level language. Hence it can be used to create system softwares and application softwares.

- Programs written in C are efficient and fast. This is because of its variety of data types and powerful operators.
- Along with the 32 keywords C has plenty of built-in functions which can be used for developing programs.
- C is highly portable. This means that C programs written for one computer can be run on another computer with very minimal modifications.
- C has the ability to extend itself. We can continuously add functions to C library. With the availability of a large number of functions, the programming task becomes easier.
- C is a structured programming language. So the user can think of a problem in terms of function modules or blocks. A collection of such function modules makes a complete program. This modular structure makes program debugging and testing easier.

2.3 Basic Structure of C Programs

```

Documentation Section
Link Section
Definition Section
Global Declaration Section
main() Function Section
{
    Declaration Section
    Executable Part
}
Sub Program Section

Function 1
Function 2
.           (User Defined Functions)
.
Function n

```

Listing 2.1: Structure of C Program

- The Documentation Section consists of a set of comment lines giving the name of the program, the author and other details.
- Link Section provides instructions to the compiler to link functions from the system library.
- The Definition Section defines all symbolic constants.
- Some variables are used in more than one functions. Such variables are called **global** variables. They are declared in the Global Declaration Section.
- Every C program must have a one `main()` function section. This section contains two parts :
 - Declaration Section declares all the variables used in the executable part
 - Executable Part contains program statements.
 These two parts should be enclosed between opening and closing braces. The program execution begins at the opening brace and ends at the closing brace. All the statements in the declaration and executable parts end with a semicolon.
- The Subprogram Section contains all the user defined functions that are called in the main function.

2.4 Executing a C Program

There are 4 steps in executing a C program.

1. Create the program
2. Compile the program
3. Link the program with functions that are needed from the C library

4. Execute the program

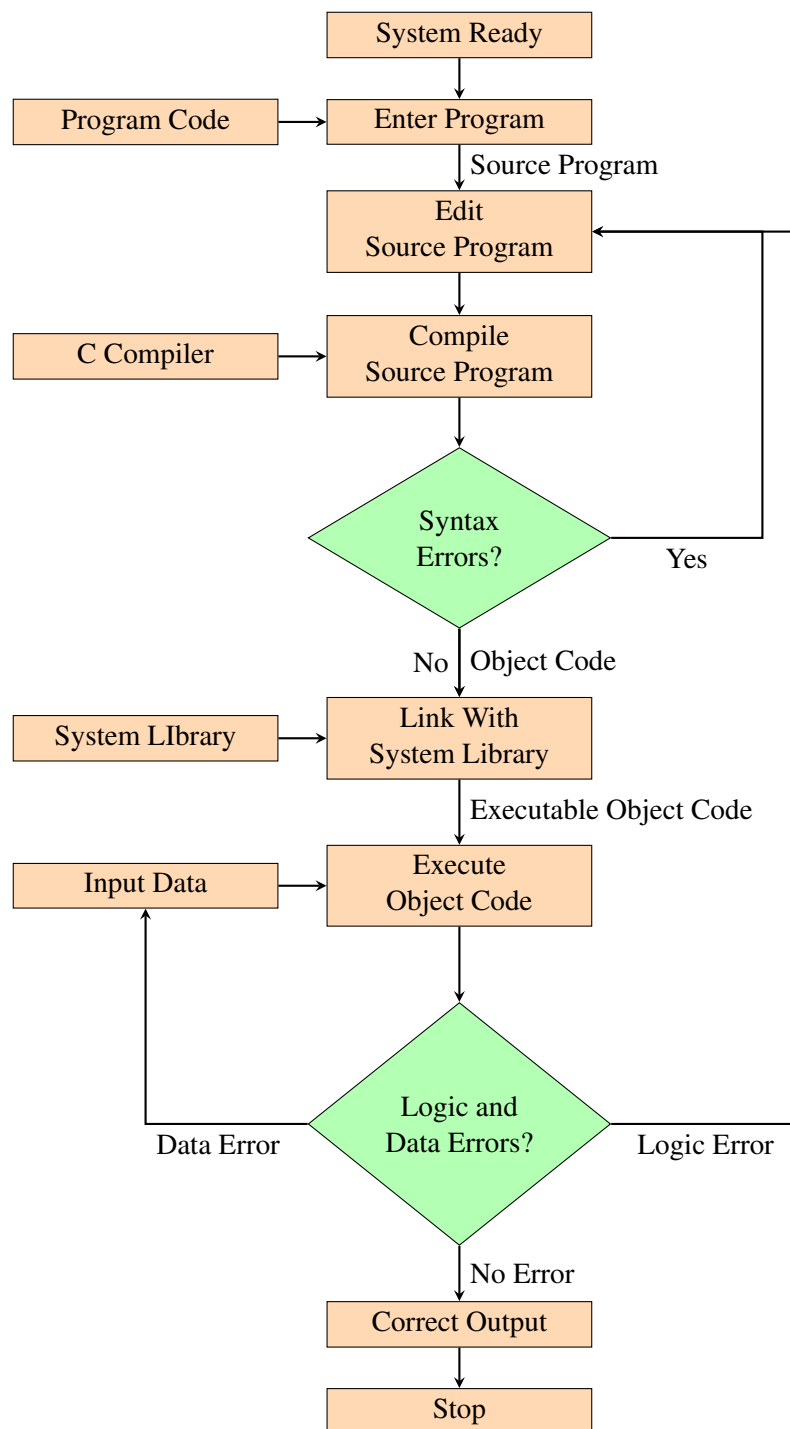


Figure 2.1: Process of compiling and running a C program

2.5 Source Code, Object Code & Executable File

Source code and object code refer to the “before” and “after” versions of a computer program that is compiled before it is ready to run in a computer. The **source code** consists of the programming statements that are created by a programmer with a text editor and then saved in a file. For example,

a programmer using the C language types in C language statements using a text editor and then saves them as a named file. This file is said to contain the source code.

It is now ready to be compiled with a C compiler and the resulting output, the compiled file, is referred to as **object code**. The object code file contains a sequence of instructions that the processor can understand but that is difficult for a human to read or modify. Object code is a portion of machine code that hasn't yet been linked into a complete program.

Object files can in turn be linked to form an executable file. An **executable file** is a computer file that contains instructions in a form that a computer's operating system or application can understand and follow. Computers must use executable files to carry out the tasks that you give to them. Every application you run starts off with an executable program.

2.6 File Extension

A filename extension is a suffix (separated from the base filename by a dot or space) to the name of a computer file. It indicates the encoding (file format) of its contents. Examples of filename extensions are .png, .jpeg, .exe, .mp3, .avi and .txt.

2.7 Character Set

Characters in C are grouped into following categories:

- Letters
 - Uppercase A . . . Z
 - Lowercase a . . . z
- Digits
 - All decimal digits 0 . . . 9
- Special Characters

,	comma	&	ampersand	.	period	~	caret
;	semicolon	*	asterisk	:	colon	-	minus sign
?	question mark	+	Plus sign	'	apostrophe	<	Less than
"	quotation mark	>	Greater than	!	exclamation mark		vertical bar
(	left parenthesis	/	slash	)	right parenthesis	\	backslash
~	tilde	[	left bracket	_	underscore	]	right bracket
\$	dollar sign	{	left brace	%	percentage sign	}	right brace
#	hash symbol						

- White Spaces
 - Blank space
 - Horizontal tab
 - Carriage return
 - New line
 - Form feed

2.8 C Tokens

In a C program smallest individual units are called C tokens. C has 6 types of tokens

1. **Keywords:** float, while, for, if etc.
2. **Identifiers:** main, amount etc.
3. **Constants:** 100, 15.5 etc.

4. **Strings:** "INDIA", "cat" etc.
5. **Special Symbols:** [], {} etc.
6. **Operators:** +, -, *, / etc.

2.9 Keywords

Keywords have fixed meanings. They are the basic building blocks for program statements. All keywords must be written in lower case letters. There are 32 keywords in C.

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Table 2.2: ANSI C Keywords

2.10 Identifiers

Identifiers refers to the names of variables, functions and arrays. These are user defined names and consists of a sequence of letters and digits with a letter as a first character. Both upper case and lower case letters are permitted. The underscore character is also permitted in identifiers.

Rules for identifiers are:

1. First character must be an alphabet (or underscore)
2. Must consist of only letters, digits or underscores
3. Only first 31 characters are significant
4. Keywords cannot be used as identifiers
5. Must not contain white space
6. Identifiers are case sensitive. Therefore the identifier `area` is different from the identifier `Area`.

2.11 Constants

Constants in C refers to fixed values that do not change during the execution of a program. C supports several types of constants:

- Numeric Constants
 - Integer Constants
 - Real Constants
- Character Constants
 - Single Character Constants
 - String Constants

2.11.1 Integer Constants

- Integer constant refers to a sequence of digits.
- There are three types of integer constants:
 - **Decimal Integer Constants:** Consists of a sequence of digits, 0 through 9 preceded by an optional - or + sign. The largest integer that can be stored is machine-dependent.

It is 32767 in a 16-bit machine. It is also possible to store larger integer constants by appending qualifiers such as `U`, `L`, `UL` to the constants. e.g. `123`, `-321`, `56789L`.

- **Octal Integer Constants:** Consists of any combination of digits from 0 through 7, with a leading 0. e.g. `037`, `0123`.
- **Hexadecimal Integer Constants:** They may include alphabets A through F or a through f. The letters A through F represents numbers 10 through 15. These constants should be preceded by `0x`. e.g. `0x2`, `0x2A`

2.11.2 Real Constants

- Numbers containing fractional parts are called real constants. e.g. `0.083`, `0.45`.
- A real number may also be represented in exponential (scientific) notation. e.g. `145.67` may be represented as `1.4567e2`. Here `e2` stands for multiply by 10^2

The general form is **mantissa e exponent**. The mantissa is either a real number expressed in decimal notation or an integer. The exponent is an integer number with an optional plus or minus sign.

2.11.3 Single Character Constants

- A single character constant contains a single character enclosed within a pair of single quotation marks. e.g. `'5'`, `'x'`, `' '`
- Character constants have integer values known as ASCII values
For example,

```
printf("%d", 'a');
```

would print 97, the ASCII value of the letter a.
Similarly,

```
printf("%c", 97);
```

would print a.

2.11.4 String Constants

- A string constant is a sequence of characters enclosed within a pair of double quotation marks. The characters may be letters, special characters or blank space. e.g. `"India"`, `"Good Morning"`.

2.11.5 Backslash Character Constants

C has many backslash character constants that are used in output functions.

Constant	Meaning
'\a'	audible alert (bell)
'\b'	backspace
'\f'	form feed
'\n'	new line
'\r'	carriage return
'\t'	horizontal tab
'\v'	vertical tab
'\''	single quote
'\"'	double quote
'\?'	question mark
'\\'	backslash

Table 2.3: Backslash Character Constants

2.12 Variables

- Variables are names used to refer to some location in memory that holds a value with which we are working.
- Unlike constants, that remain unchanged during execution of a program, a variable may take different values at different stages of execution
- A variable name must be given in such a way that it is meaningful in the context of the program. e.g. `average`, `sum`, `weight`.
- Rules for creating variable names:
 - Variable names may consists of letters, digits and the underscore (`_`) character
 - They must begin with a letter
 - Variable name length should be limited to 31 characters
 - Uppercase and lowercase are significant. For example, `Sum` and `sum` are different variables
 - It should **not** be a keyword
 - White spaces are not allowed

2.13 Data Types

Data types are keywords which are used for assigning a type to a variable. ANSI C supports 3 classes of data types:

- Primary (Fundamental) data types
- Derived data types
- User-defined data types

Examples of Derived data types such as arrays, structures, unions, pointers will be discussed later.

2.13.1 Primary(Fundamental) Data Types

All C compilers support 5 primary data types, namely, integer (`int`), character (`char`), floating point (`float`), double precision floating point (`double`), and `void`.

Integer Types

- Integers are whole numbers with a range of values supported by a particular machine.
- Generally integers occupy one word of a storage, and since the word size of machines vary (16, 32 or 64 bits), size of an integer that can be stored depends on the computer.

- In a 16 bit machine, a signed integer uses 1 bit for sign and 15 bits for magnitude of the number
- Unsigned integers use all the bits for magnitude of the number and will always be positive
- C has three classes of integers, namely `short int`, `int` and `long int` in both signed and unsigned forms.
- `short int` is used when dealing with fairly small integers where as `long int` is used when dealing with large numbers.

Floating Point Types

- Floating point numbers are stored in 32 bits with 6 digits of precision
- Floating point numbers are defined in C by the keyword `float`
- A `double` data type number uses 64 bits giving a precision of 14 digits

Character Types

- A single character can be defined as a **character(char)** type data
- Characters are usually stored in 8 bits

Void Types

- A `void` type has no values
- Usually used to specify the type of functions
- The type of a function is said to be `void` when it does not return any value to the calling function

Type	Size (bits)	Range
char OR signed char	8	-128 to 127
unsigned char	8	0 to 255
int OR signed int	16	-32768 to 32767
unsigned int	16	0 to 65535
short int OR signed short int	8	-128 to 127
unsigned short int	8	0 to 255
long int OR signed long int	32	-2147483648 to 2147483647
unsigned long int	32	0 to 4294967295
float	32	3.4E-38 to 3.4E+38
double	64	1.7E-308 to 1.7E+308
long double	80	3.4E-4932 to 1.1E+4932

Table 2.4: Size and Range of data types on a 16 bit machine

2.14 Declaration of Variables

Variable declaration does 2 things:

- It tells the compiler what the variable name is
- It specifies what type of data the variable will hold

The declaration of variable must be done before they are used in the program

2.14.1 Primary Type Declaration

The syntax for declaring a variable is as follows:

```
data-type v1, v2, . . . . vn;
```

Listing 2.2: Syntax for variable declaration

$v_1, v_2, \dots, v_n$ are the names of the variables. Variables are separated by commas. A declaration statement must end with a semicolon. For example:

```
int sum;
int area, volume;
double amount;
```

Listing 2.3: Example for variable declaration

2.14.2 User-Defined Type Declaration

- C supports a feature known as **type definition** that allows users to define an identifier that would represent an existing data type
- The user defined data type identifier can be used later to declare variable
- Syntax is:

```
typedef type identifier;
```

Listing 2.4: Syntax for type definition

Where `type` refers to an existing data type and `identifier` refers to the new name given to the data type.

Once a user defined data types is defined, `identifier` can be used instead of `type`.

```
typedef int units;
units batch1, batch2;
```

Listing 2.5: Example for type definition

Enumerated Data Types

- Another user defined data type
- It is defined as

```
enum identifier {value1, value2, .... valuen};
enum identifier v1,v2,....vn;
```

Listing 2.6: Syntax for `enum`

The `identifier` is a user-defined enumerated data type which can be used to declare variable that can have one of the values enclosed within the braces (known as enumeration constants).

- The enumerated variable $v_1, v_2, \dots, v_n$ can only have one of the values `value1, value2, ..., valuen`.

```
enum color {blue, white, yellow, black};
enum color car_color;
car_color = blue;
```

Listing 2.7: Example for enumerated data type

The compiler automatically assigns integer digits beginning with 0 to all the enumeration constants. That is, blue will have value 0, white will have value 1, yellow will have value 2 and black will have value 3.

2.15 Defining Symbolic Constants

If certain unique constants like π are required in a program, their value can be defined using symbolic constants.

```
#define PI 3.14
```

Listing 2.8: Example for symbolic constants

Advantages of using symbolic constants are

- **Modifiability:**

Sometimes it is required to change the value from 3.14 to 3.14159 to improve accuracy. In such cases it is difficult to search through the entire program and replace every 3.14 with 3.14159. If we define a constant PI, we need to change the value only where it has been defined.

- **Understandability:**

When a numeric value appears in a program, it is not always clear what it means. Assignments of such constants to symbolic constants makes things more clear.

The following rules are applied when defining symbolic constants:

- Usually symbolic names are written in capital letters to distinguish them from normal variable names
- No blank spaces are allowed between `#` and `define`
- A blank space is required between `define` and symbolic name and between symbolic name and its value
- After definition, symbolic names should not be assigned any other value within the program

2.16 Operators in C

Operators in C can be classified into following categories:

- Arithmetic Operators
- Relational Operators
- Logical Operators
- Assignment Operators
- Increment and Decrement Operators
- Conditional Operators
- Bitwise Operators
- Special Operators

2.16.1 Arithmetic Operators

C provides all the basic arithmetic operators

Operator	Description
+	Addition
-	Subtraction or unary minus
*	Multiplication
/	Division
%	Modulus (Gives remainder)

- Integer division truncates any fractional part. i.e. $9/2$ will give 4.
- Modulus operation gives the remainder of an integer division. This operation can be applied **only** on integer values. e.g. $9\%4$ will give 1.

Integer Arithmetic

When both the operands of an arithmetic expression are integers, the expression is called an **integer expression** and the operation is called **integer arithmetic**. Integer arithmetic always yields an integer value.

If $a = 20$ and $b = 3$

$$a - b = 17$$

$$\begin{aligned}a + b &= 23 \\ a / b &= 6 \\ a \% b &= 2\end{aligned}$$

During modulus operation, the sign of the result is always the sign of the first operand (the dividend).

$$\begin{aligned}-14 \% 3 &= -2 \\ -14 \% -3 &= -2 \\ 14 \% -3 &= 2\end{aligned}$$

Real Arithmetic

An arithmetic operation involving only real operands are called real arithmetic. A real arithmetic will always yield a real result.

If x is a **float** then:

$$x = 6.0/7.0 = 0.857143$$

Mixed-mode Arithmetic

When one of the operands is real and the other is integer, then the arithmetic operation is called a mixed mode arithmetic. In this case, the the result will always be real.

$$15/10.0 = 1.5$$

2.16.2 Relational Operators

Relational Operators are used to compare two values. An expression containing relational operators is termed as a **relational expression**. The value of a relational expression is either 1 or 0. If it is 1, then the specified relation is true and if its is 0, then the specified relation is false. C Supports 6 relational operators:

Operator	Description
<	is less than
<=	is less than or equal to
>	is greater than
>=	is greater than or equal to
==	is equal to
!=	is not equal to

A simple relational expression uses only one relational operator and takes the following form

```
ae1 relational operator ae2
```

Where ae1 and ae2 are arithmetic expressions, which may be simple constants, variables or combination of them.

```
4.5 <= 10    //True
10 < 7 + 5    //False
```

Listing 2.9: Example for relational expressions

When arithmetic expressions are used on either side of a relational operator, then the arithmetic expression will be evaluated first and then the results are compared. That is arithmetic operations have higher priority over relational operator.

2.16.3 Logical Operators

C has 3 logical operators:

- `&&` meaning logical AND
- `||` meaning logical OR
- `!` meaning logical NOT

The logical operators `&&` and `||` are used when we want to test more than one condition and make decision. An example is:

```
a > b && x==10
```

Listing 2.10: Example for logical operator

An expression that combines two or more relational expressions is termed as **logical expression**. A logical expression also yields a value 1 or 0 according to the following truth tables:

op-1	op-2	op-1 && op-2
Non-zero	Non-zero	1
Non-zero	0	0
0	Non-zero	0
0	0	0

Table 2.5: Truth Table for AND(&&) operation

op-1	op-2	op-1 op-2
Non-zero	Non-zero	1
Non-zero	0	1
0	Non-zero	1
0	0	0

Table 2.6: Truth Table for OR(||) operation

op	!op
Non-zero	0
0	1

Table 2.7: Truth Table for NOT(!) operation

2.16.4 Assignment Operators

Assignment operators are used to assign the result of an expression to a variable. In addition to the usual assignment operation (`=`), C has a set of **shorthand** assignment operators of the form:

```
v op= exp;
```

Where `v` is a variable, `exp` is an expression and `op` is a C binary arithmetic operation. The assignment statement

```
v op= exp;
```

is equivalent to

```
v = v op exp;
```

For example:

Statement with Shorthand assignment	Equivalent operation
<code>x += y + 1;</code>	<code>x = x + (y+1);</code>
<code>x += 3;</code>	<code>x = x + 3;</code>
<code>a *= a;</code>	<code>a = a * a;</code>
<code>a /= n + 1</code>	<code>a = a / (n+1);</code>

2.16.5 Increment (++) and Decrement (--) Operators

- The increment and decrement operators are unary operator, meaning they operate on a single operand
- The increment operator (++) adds 1 to the operand while decrement (--) operator subtracts 1
- Both these operators can take two forms: **prefix** or **postfix**.

- **Prefix:** In prefix form, the increment or decrement operator comes before the operand. For example:

```
++m;
--m;
```

Listing 2.11: Example for prefix form

- **Postfix:** In postfix form, the increment or decrement operator comes after the operand. For example:

```
m++;
m--;
```

Listing 2.12: Example for postfix form

`++m` and `m++` mean the same thing when they are independent statements. But they behave differently when they are used in expressions on the right hand side of an assignment statement. For example:

```
m = 5;
y = ++m;
```

In this case, the value of `m` and `y` will be 6.

But if we rewrite the above statements as

```
m = 5;
y = m++;
```

The value of `y` would be 5 and `m` would be 6.

Rules for ++ and -- operator

- Increment and decrement operators are unary operators and they require a single variable as there operand
- When postfix ++ (or --) is used with a variable in an expression, the expression is evaluated first using the original value of the variable and then the variable is incremented (or decremented) by 1.
- When prefix ++ (or --) is used in an expression, the variable is incremented (or decremented) first and then the expression is evaluated using the new value of the variable.

2.16.6 Conditional Operator

A ternary operator (operator with 3 operands), `?:` is available in C to construct conditional expressions of the form

```
exp1 ? exp2 : exp3;
```

Conditional operator works as follows

- `exp1` is evaluated first. If it is nonzero (true), then the expression `exp2` is evaluated and becomes the value of the entire expression.
- If `exp1` is 0 (false), `exp3` is evaluated and its value becomes the value of the entire expression.

```
a = 10;
b = 15;
x = (a > b) ? a : b;
```

Listing 2.13: Example for conditional operator

In this example, `x` will be assigned the value of `b`.

2.16.7 Bitwise Operators

Bitwise operators manipulate data at the bit level.

Bitwise AND (&)

Bitwise AND operation is represented by a single ampersand (&) and it operates on two integer expressions. The result of ANDing operation is 1 if both the bits have a value of 1, otherwise it is 0. For example, Let `x` and `y` be two variable with values 13 and 25. The binary representation of these two variable are

```
x: 0000 0000 0000 1101
y: 0000 0000 0001 1001
```

If the following statement is executed

```
z = x & y;
```

Then the result would be `z = 9`.

```
z: 0000 0000 0000 1001
```

Bitwise OR

Bitwise OR operation is represented by a single pipe symbol (|) and it operates on two integer expressions. The result of ORing operation is 1 if at least one of the bits have a value of 1, otherwise it is 0. For example, Let `x` and `y` be two variable with values 13 and 25. The binary representation of these two variable are

```
x: 0000 0000 0000 1101
y: 0000 0000 0001 1001
```

If the following statement is executed

```
z = x | y;
```

Then the result would be `z = 29`.

```
z: 0000 0000 0001 1101
```

Bitwise Exclusive OR

Bitwise exclusive OR is represented by the symbol ^ . Result of exclusive OR is 1 if only one of the bits is 1, otherwise 0. For example, Let `x` and `y` be two variable with values 13 and 25. The binary representation of these two variable are

```
x: 0000 0000 0000 1101
y: 0000 0000 0001 1001
```


If the following statement is executed

```
z = x ^ y;
```

Then the result would be $z = 20$.

z: 0000 0000 0001 0100

Bitwise Shift Operators

The shift operators are used to move bit patterns either to the left or to the right.

- Left shift: `op << n;`
- Right shift: `op >> n;`

`op` is the integer expression that is to be shifted and `n` is the number of bit positions to be shifted.

The left shift operation causes all the bits in the operand `op` to be shifted to the left by `n` positions. The leftmost `n` bits in the original bit pattern will be lost and the rightmost `n` bit positions that are vacated will be filled with 0s.

Similarly the right shift operation causes all the bits in the operand `op` to be shifted to the right by `n` positions. The rightmost `n` bits in the original bit pattern will be lost. The leftmost `n` bit positions that are vacated will be filled with 0s if operand `op` is an unsigned integer.

If

x: 0100 1001 1100 1011

Then

```
x << 3 = 0100 1110 0101 1000
x >> 3 = 0000 1001 0011 1001
```

Bitwise Complement Operator

The complement operator `~` (tilde) is a unary operator and it inverts all the bits represented by its operand, i.e., 0s become 1 and 1s become 0. An example is given below.

```
x = 1001 0110 1100 1011
~x = 0110 1001 0011 0100
```

2.16.8 Special Operators

The `sizeof` Operator

The `sizeof` operator returns the number of bytes occupied by its operand. The operand may be a variable, a constant or a data type. An example is given below.

```
m = sizeof(sum);
n = sizeof(int);
k = sizeof(235L);
```

The Comma (,) Operator

Comma operator is used to link related expressions together. A comma linked list of expressions are evaluated from left to right and the value of the right most expression is the value of the combined expression. An example is given below.

```
value = (x = 10, y = 5, x + y);
```

The above statement first assigns value 10 to `x`, then assigns 5 to `y` and finally assigns 15 to `value`.

Comma operator is most commonly used in `for` statement. For example:

```
for (exp1a, exp1b; exp2; exp3)
```

where `exp1a` and `exp1b` are the two expressions separated by the comma operator. These two expressions would typically initialize two separate variables. Similarly

```
for (exp1; exp2; exp3a, exp3b)
```

where `exp3a` and `exp3b` are the two expressions separated by the comma operator. These two expressions would typically alter two separate variables that are used simultaneously within the loop.

2.17 Type Conversion in Expressions

2.17.1 Implicit Type Conversion

C permits mixing of constants and variable of different types in an expression. C automatically converts any intermediate values to the proper type so that the expression can be evaluated without losing any significance. This automatic conversion is known as **implicit type conversion**.

During evaluation, if the operands are of different types, the **lower** type is automatically converted to **higher** type before the operation proceeds. The result is of the higher type.

Example:

```
float x;
x = 9/2.0;
```

Here, in the expression `9/2.0`, an integer and floating point operands are present. So the **lower** type integer will be converted to float before the operation is executed. i.e. 9 will be converted to 9.0. So the expression will now become `9.0/2.0`. The result will be 4.5 which will be stored in the floating point variable `x`.

Conversion hierarchy is as shown below with the highest data type at the left.

long double - double - float - unsigned long int - long int - unsigned int - int - char

2.17.2 Explicit Type Conversion

In some cases we may want to force type conversion in a way that is different from the automatic conversion. In such situation we can use explicit type conversion which is done as follows.

```
(type-name) expression;
```

where type-name is one of the standard C data types. The expression may be a constant, variable or an expression.

Example	Action
<code>x = (int)7.5;</code>	7.5 is converted 7 and stored in x.
<code>a = (int)21.3/(int)4.5</code>	Evaluated as 21/4 and result would be 5

2.17.3 Operator Precedence and Associativity

- Each operator in C has a precedence associated with it.
- This precedence is used to determine how an expression involving more than one operator is evaluated.
- The operators at higher level of precedence are evaluated first.
- The operators at the same level are evaluated from left to right or from right to left, depending on the level. This is known as associativity property of an operator.

Operator	Description	Associativity	Rank
() []	Function Call Array element reference	Left to right	1
+ - ++ -- ! ~ * & sizeof (type)	Unary plus Unary minus Increment Decrement Logical negation Ones complement Pointer reference Address Size of an object Type cast	Right to left	2
* / %	Multiplication Division Modulus	Left to right	3
+ -	Addition Subtraction	Left to right	4
<< >>	Left shift Right shift	Left to right	5
< <= > >=	Less than Less than or equal to Greater than Greater than or equal to	Left to right	6
== !=	Equal to Not equal to	Left to right	7
&	Bitwise AND	Left to right	8
^	Bitwise XOR	Left to right	9
	Bitwise OR	Left to right	10
&&	Logical AND	Left to right	11
	Logical OR	Left to right	12
?:	Conditional expression	Right to left	13
= *=/, %= +,-, &= ^=, = <<=, >>=	Assignment operators	Right to left	14
,	Comma operator	Left to right	15

Rules of Precedence and Associativity

- Precedence rules decides the order in which different operators are applied
- Associativity rule decides the order in which multiple occurrences of the same level operator are applied

Example:

Let $a = 9$, $b = 12$ and $c = 3$. Then evaluate $x = a - b/3 + c * 2 - 1$

$$x = 9 - 12/3 + 3 * 2 - 1$$

This can be evaluated as follows:

- **First pass:**
 - Step 1:** $x = 9 - 4 + 3 * 2 - 1$
 - Step 2:** $x = 9 - 4 + 6 - 1$
- **Second pass:**
 - Step 3:** $x = 5 + 6 - 1$
 - Step 4:** $x = 11 - 1$
 - Step 5:** $x = 10$

3. Program Control Statements

3.1 Managing Input and Output

C does not have any built-in input/output statements. All input/output functions are carried out through functions such as `printf()` and `scanf()`. Every program that uses standard input/output functions must contain the following statement at the beginning:

```
#include <stdio.h>
```

- The filename `stdio.h` stands for standard input-output header file.
- The instruction `#include <stdio.h>` tells the compiler to search for a file named `stdio.h` and place the contents at this point in the program.

3.1.1 Reading a Character

Reading a single character can be done using the function `getchar()`. It takes the following form:

```
variable_name = getchar();
```

where `variable_name` is a valid C name that has been declared as `char` type. When this statement is encountered, the computer waits until a key is pressed and then assigns this character as a value to `getchar()` function. This value will then be assigned to the variable name on the left hand side. For example:

```
char name;  
name = getchar();
```

will assign the character 'H' to the variable `name` when key 'H' is pressed on the keyboard.

3.1.2 Writing a Character

The function `putchar()` writes characters one at a time to the terminal (monitor). It takes the following form:

```
putchar(variable_name);
```

where `variable_name` is a `char` type variable containing a character. An example is given below:

```
char answer;
answer = 'Y';
putchar(answer);
```

will display the character 'Y' on the screen.

3.1.3 Formatted Input

Formatted input refers to an input that has been arranged in a particular format. `scanf()` function is used to read formatted input data. The general form is:

```
scanf("control string", arg1, arg2, . . . . argn);
```

The arguments `arg1`, `arg2`,...`argn` specify the address of locations where the data is stored.

Control string (format string) contains the following:

- Field (or format) specification, consisting of the conversion character %, a datatype character (or type specifier) and an optional number specifying the field width.
- Blanks or tabs or newlines

Blanks, tabs and newlines are ignored. The data type character indicates the type of data that is to be assigned to the variable associated with the corresponding argument.

Inputting Integer Numbers

Field specification for reading an integer number is

```
%wd
```

`w` is an integer number that specifies the field width of the number to be read and `d`, known as data type character specifies that the number to be read is an integer. For example:

```
scanf("%2d%5d",&num1,&num2);
```

Suppose the input data is as follows:

```
50 31426
```

The value 50 will be assigned to `num1` and 31426 to `num2`.

Suppose the input data is as follows:

```
31426 50
```

The value 31 will be assigned to `num1` and 426 to `num2`. This is because `%2d` specifies that only two digits are to be read into `num1`. This kind of errors can be eliminated by using field specification without the field width specification. For example:

```
scanf("%d%d",&num1,&num2);
```

Inputting Real Numbers

In case of real numbers field width need not be specified. `scanf()` reads real numbers using the simple specification `%f` for both normal and scientific notation. For example:

```
scanf("%f%f%f",&x,&y,&z);
```

with the input data

```
475.89 43.21E-1 678
```

will assign the values 475.89 to `x`, 4.321 to `y` and 678.0 to `z`.

Inputting Character Strings

`scanf()` can be used to read input strings containing more than one character. Following are the specifications for reading character strings.

```
scanf("%ws",arg);
```

A single character can be read using the field specification `%c`.

Reading Mixed Data Types

```
scanf("%d%c%f",&count,&code,&ratio);
```

can be used to read the input data

```
15 p 1.575
```

3.1.4 Formatted Output

`printf()` function can be used to display outputs. `printf()` provides certain features that can be used to control the alignment and spacing of print-outs on the screen. The general form of `printf()` statement is:

```
printf("control string",arg1,arg2,... argn);
```

Control string consists of three types of items:

- Characters that will be printed on the screen as they appear
- Format specifications that define the output format for display of each item
- Escape sequence characters such as `\n`, `\t` etc.

The control string indicates how many arguments follow and what their types are. The arguments `arg1`, `arg2`,...`argn` are the variables whose values are formatted and printed according to the specifications of the control string. The arguments should match in number, order and type with the format specifications.

Output of Integer Number

Format specification for printing an integer number is:

```
%wd
```

where `w` specifies the minimum field width for the output. `d` specifies that the value to be printed is an integer. The number is written right justified in the given field width. Leading blanks will appear as necessary. Some examples are given below:

```
printf("%d", 9876);
printf("%6d", 9876);
printf("%2d", 9876);
printf("%-6d%-6d", 9876,7456);
printf("%06d", 9876);
```

Listing 3.1: Examples for Formatted Integer Output

Output of the above statements will be as follows

```
9876
 9876
9876
9876 9876
009876
```

Output of Real Numbers

Format specification is:

```
%w.pf
```

`w` indicates the minimum number of positions that are to be used for the display of the value and `p` indicates the number of digits to be displayed after the decimal point (precision). Some examples are given below:

```

y = 98.7654
printf("%7.4f", y);
printf("%7.2f", y);
printf("%-7.2f", y);
printf("%f", y);
printf("%e", y);
printf("%10.2e", y);

```

Listing 3.2: Examples for Formatted Real Output

Output of the above statements will be as follows

```

98.7654
 98.77
98.77
98.7654
9.876540e+01
 9.88e+01

```

Printing of a Single Character

Format specification is %wc. The character will be displayed right justified in the field of w columns. The default value of w is 1.

```

char y = 'w';
printf("%c", y);

```

Listing 3.3: Example for Formatted Character Output

Printing of Strings

Format specification is %w.ps.

where w specifies the field width for display and p instructs that only the first p characters of the string are to be displayed. The display is right justified.

Some examples are given below:

```

char place[50] = "NEW DELHI 110001";
printf("%s\n", place);
printf("%20s\n", place);
printf("%20.10s\n", place);

```

Listing 3.4: Examples for Formatted String Output

In this example, the string "NEW DELHI 110001", containing 16 characters (including blanks). Output of the above example will be

```

NEW DELHI 110001
  NEW DELHI 110001
    NEW DELHI

```

3.2 Decision Making and Branching

Decision making and branching statements in C are used to change the order of execution of statements.

- if statement
- switch statement
- Conditional Operator (? :) statement
- goto statement

3.2.1 if Statement

General form is:

```
if (test expression)
```

Listing 3.5: Syntax for if Statement

It allows the computer to evaluate the expression first and then, depending on whether the value of the expression is **true** (non zero) or **false** (zero), it transfers the control to a particular statement. At this point, the program has two paths to follow. One for the **true** condition and the other for the **false** condition as shown below.

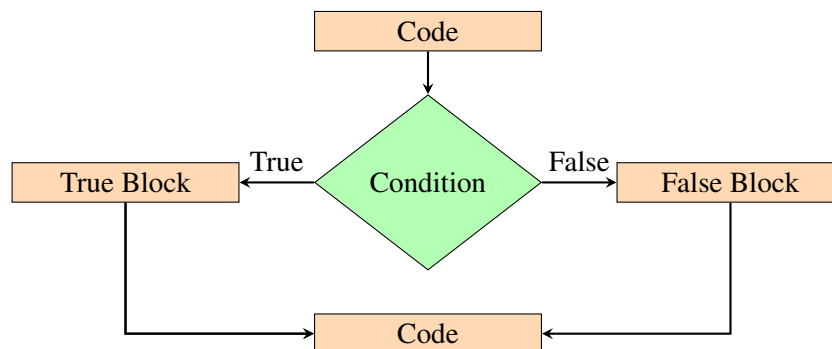


Figure 3.1: Flowchart for if Statement

The if statement may be implemented in different forms depending on the complexity of conditions to be tested. The different forms are:

1. Simple if statement
2. if...else statement
3. Nested if..else statement
4. else if ladder

Simple if statement

The general form of a simple if statement is

```
if(test_expression)
{
    statement_block;
}
statement_x;
```

The `statement_block` may be a single statement or a group of statements. If the `test_expression` is true, the `statement_block` and `statement_x` will be executed; otherwise the `statement_block` will be skipped and the execution will jump to the `statement_x`.

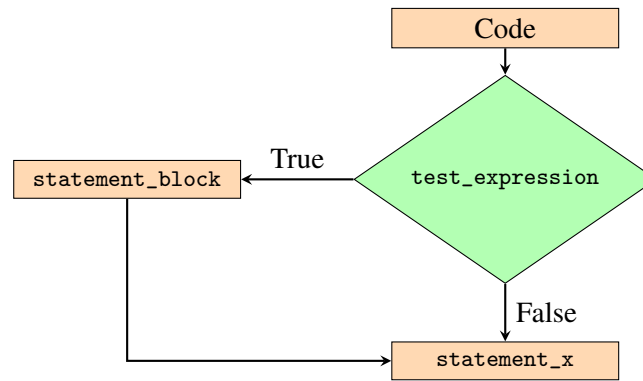


Figure 3.2: Flowchart of simple if statement

An example is given below

```

if (age > 60)
{
    amount = amount + 100;
}
printf("Total Amount= %d", amount);
  
```

In this example if the age is greater than 60, then a bonus of 100 will be added to the total amount.

The if...else Statement

The general form of if...else statement is

```

if (test_expression)
{
    true_block_statements;
}
else
{
    false_block_statements;
}
statement_x;
  
```

If the test_expression is true, then the true_block_statements are executed followed by the statement_x; otherwise false_block_statements are executed followed by the statement_x.

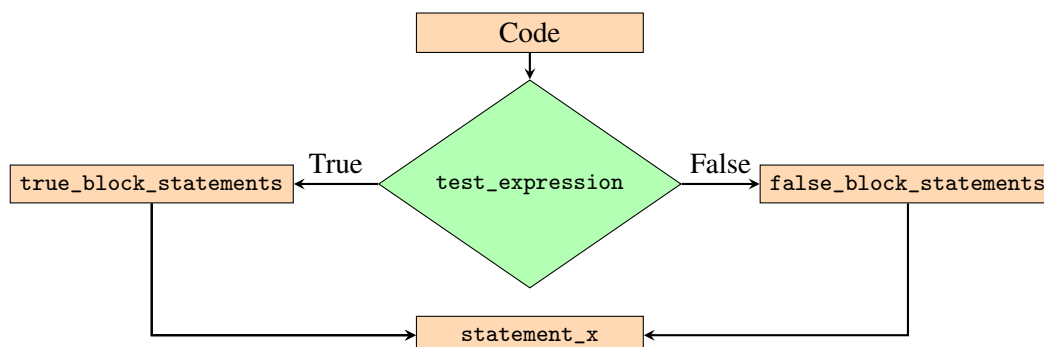


Figure 3.3: Flowchart for if...else Statement

An example is given below

```
if(n % 2 == 0)
{
    printf("The number is even");
}
else
{
    printf("The number is odd");
}
```

Nested if...else

If more than one decisions are to be made, if else statements can be nested as shown below:

```
if (test_expresssion_1)
{
    if(test_expresssion_2)
    {
        statement_block_1;
    }
    else
    {
        statement_block_2;
    }
}
else
{
    statement_block_3;
}
statement_x;
```

If test_expresssion_1 is false, the statement_block_3 will be executed followed by statement_x. Otherwise it performs the second test. If test_expresssion_2 is true, statement_block_1 will be executed; otherwise statement_block_2 will be executed and then the control is transferred to statement_x.

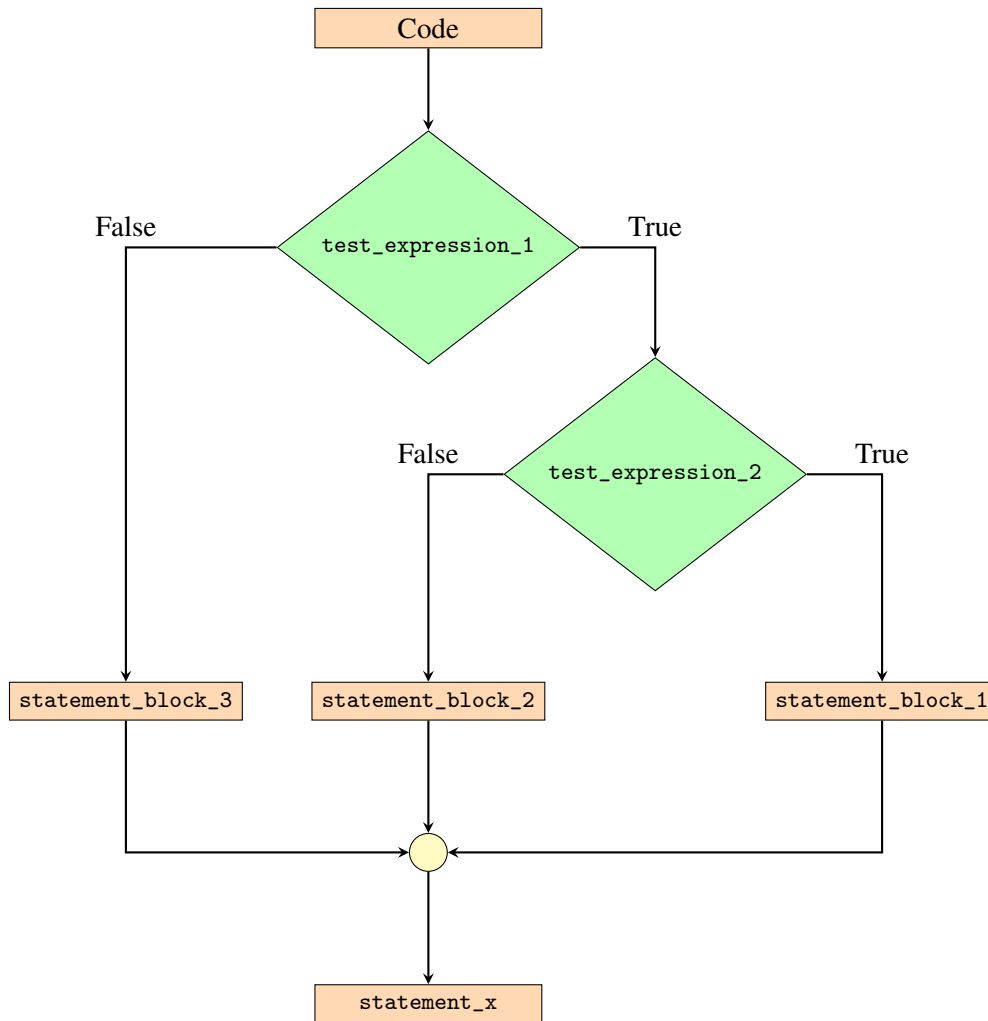


Figure 3.4: Flowchart of nested if control

An example is given below

```
if(n > 0)
{
    if(n % 2 == 0)
    {
        printf("The number is a positive even number");
    }
    else
    {
        printf("The number is a positive odd number");
    }
}
else
{
    printf("The number is negative");
}
```

Dangling else problem: One of the problem while using **nested if** is dangling else problem. This occurs when a matching if is not available for an **else**.

The else if Ladder

Else if ladder the following form:

```
if(test_expression_1)
{
    statement_block_1;
}
else if(test_expression_2)
{
    statement_block_2;
}
else if(test_expression_3)
{
    statement_block_3;
}
.
.
.
else if(test_expression_n)
{
    statement_block_n;
}
else
{
    default_block;
}
statement_x;
```

The conditions are evaluated from top to bottom. As soon as a true condition is found, the statements associated with it are executed and the control is transferred to the `statement_x` skipping the rest of the ladder. When all conditions become false, then the final `else` containing the default statement will be executed. An example is given below:

```
if(marks > 80)
{
    printf("Distinction");
}
else if(marks > 60)
{
    printf("First Class");
}
else if(marks > 40)
{
    printf("Second Class");
}
else
{
    printf("Failed");
}
```

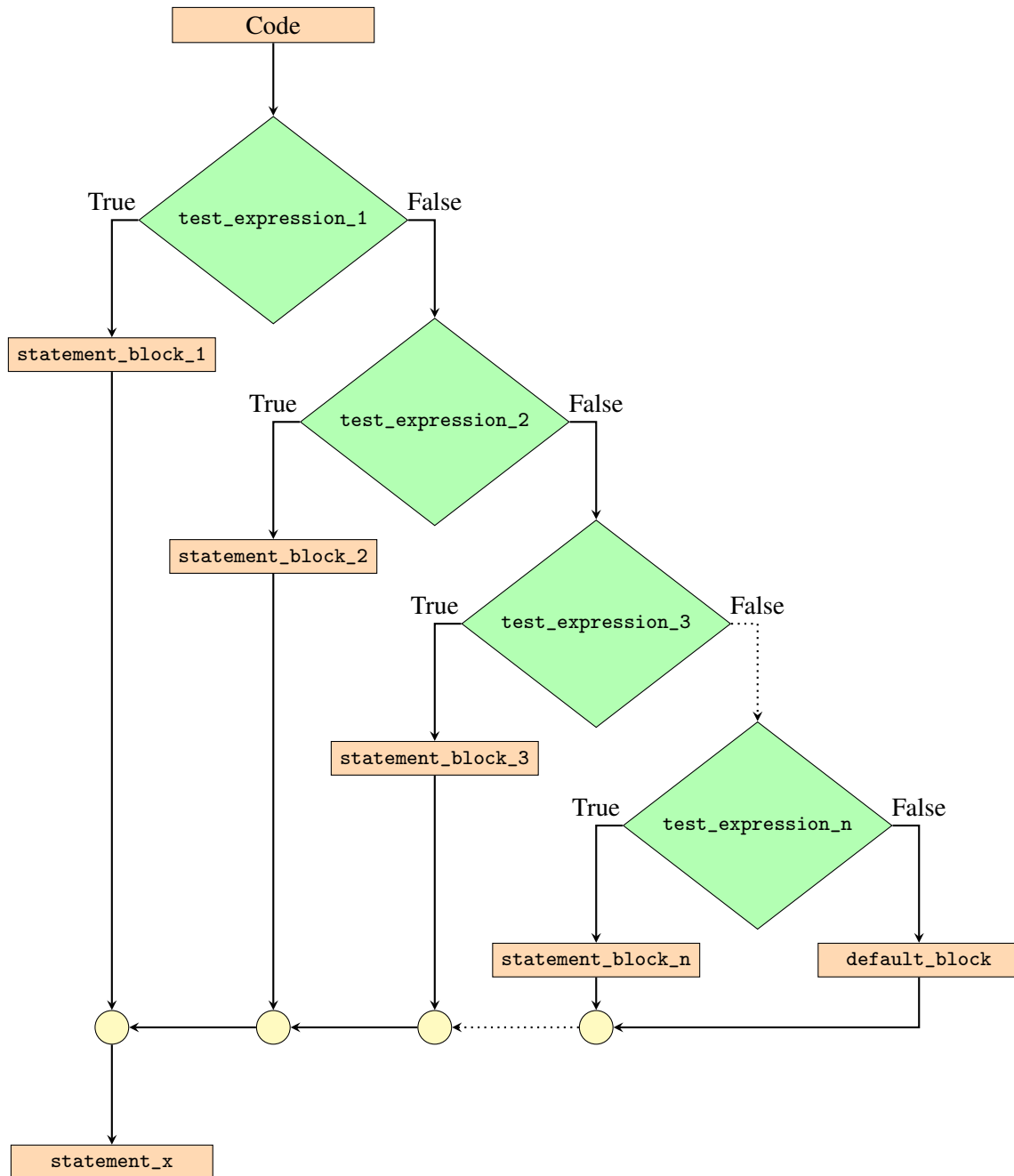


Figure 3.5: Flowchart of else if ladder

3.2.2 The `switch` Statement

The `switch` statement tests the value of a given variable against a list of `case` values and when a match is found, a block of statements associated with that `case` is executed. The general form is:

```

switch(expression)
{
    case value1:
        block1;
        break;
    case value2:
        block2;

```

```
        break;
    ....
    ....
    default:
        default_block;
}
statement_x;
```

Listing 3.6: Syntax for `switch` statement

- The expression must be an integer expression or a character.
- value1, value2,... are constants or constant expressions (evaluable to an integral constant) and are known as **case labels**. Case labels must be unique.
- block1, block2,... are statement lists

When the `switch` statement is executed, the value of the expression is compared against the values value1, value2,..... If a case is found whose value matches with the value of the expression, then the block of statements that follows the case are executed.

The `break` statement at the end of each block indicates the end of the particular case and causes an exit from the `switch` statement, transferring the control to the statement following the `switch`. The `break` statement is optional, i.e. two or more case labels may belong to the same statements.

The `default` is an optional case. It will be executed if the value of the expression does not match any of the case values. An example is given below:

```
index = marks /10;
switch(index)
{
    case 10:
    case 9:
    case 8:
        printf("Honours");
        break;
    case 7:
    case 6:
        printf("First Division");
        break;
    case 5:
        printf("Second Division");
        break;
    case 4:
        printf("Third Division");
        break;
    default:
        printf("Failed");
}
```

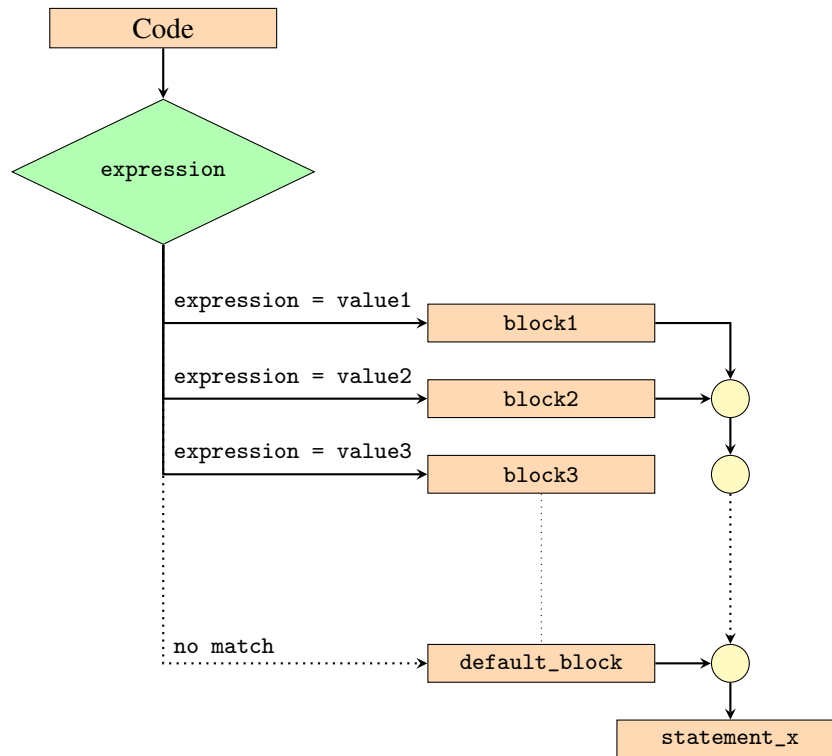


Figure 3.6: Flowchart of switch

3.2.3 The goto Statement

C program has goto statement to branch **unconditionally** from one point to another in the program. The goto requires a **label** in order to identify the place where the branch is to be made. A label is any valid variable name and must be followed by a colon. The **label** is placed immediately before the statement where the control is to be transferred. The general form is:

```
goto label;
.....
.....
.....
label:
statement;
```

The label: can be anywhere in the program either before or after the goto label; statement. If the label is before the goto statement, a loop will be formed and some statements will be executed repeatedly. Such a jump is known as backward jump. On the other hand if the label is placed after the goto statement, some statements will be skipped and the jump is known as forward jump. An example is given below:

```
void main()
{
    float x,y;
    read:
    scanf("%f", &x);
    if(x < 0)
    {
        goto read;
    }
    y = sqrt(x);
}
```



```
printf("%f %f",x,y);
goto read;
}
```

This program evaluates the square root of a series of numbers entered through the keyboard. Due to the unconditional **goto** statement at the end, the control is always transferred back to the input statement. This causes an infinite loop.

3.3 Looping

In looping, a sequence of statements are executed until some conditions for termination of the loop are satisfied. A loop consists of two segments:

- body of the loop: contains the statements to be executed repeatedly
- control statement: tests certain conditions and then directs the repeated execution of the statements in the body of the loop

Depending on the position of the control statement in the loop, a loop may be classified either as **entry-controlled loop**, or **exit-controlled loop**.

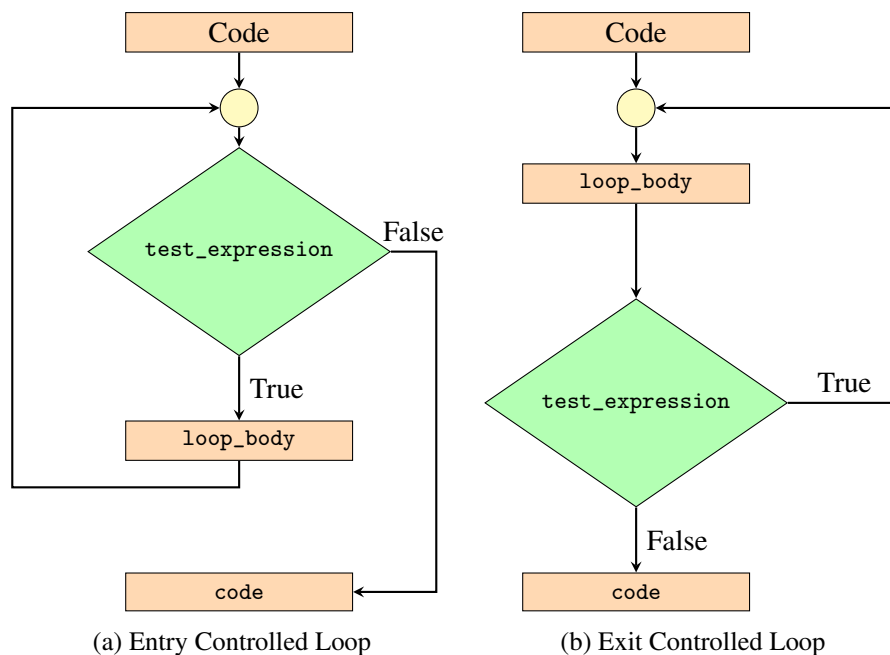


Figure 3.7: Loop Control Structures

A looping process generally involves the following four steps.

- Setting and initialization of a condition variable
- Test for a specified value of the condition variable for execution of the loop
- Execution of the statements in the loop
- Incrementing or updating the condition variable

Based on the nature of the control variable, loops may be classified into two general categories:

- **Counter-controlled loops:** When we know in advance, how many times the loop will be executed, we use a counter controlled loop.
- **Sentinel-controlled loop:** A special value called sentinel value is used to change the loop control expression from true to false. In this case, the number of times the loop will be executed is not known at the beginning.

3.3.1 The `while` statement

General form:

```
while(test_expression)
{
    loop_body
}
```

The `while` is an entry controlled loop. The test condition is evaluated and if the condition is true, then the body of the loop is executed. After execution of the body, the test condition is once again tested and if it is true, the body is executed once again. This process continues until the test condition finally becomes false. On exit, the program continues with the statement immediately after the body of the loop.

An Example is given below, which finds the sum of the numbers between 1 and 10.

```
.....
sum = 0;
n = 1;
while(n <= 10)
{
    sum = sum + n;
    n = n+1;
}
printf("Sum = %d\n", sum)
.....
```

3.3.2 The `do - while` Statement

General Form:

```
do
{
    loop_body
}while(test_expression);
```

`do-while` is an exit controlled loop. On reaching the `do` statement, the program proceeds to evaluate the body of the loop first. At the end of the loop, the test condition in the `while` statement is evaluated. If the condition is true, the program continues to evaluate the body of the loop once again. This process continues as long as the condition is true. When the condition becomes false, the loop will be terminated and the control goes to the statement that appears immediately after the `while` statement. An example is given below:

```
.....
do
{
    printf("Enter a number:");
    scanf("%d", &n);
}while(n > 0);
.....
```

This segment of program reads a number from the keyboard until a zero or a negative number is entered through the keyboard.

A comparison between `while` and `do-while` statement is given below:

while	do-while
entry controlled loop	exit controlled loop
test condition is checked at the beginning of the loop	test condition is checked at the end of the loop
loop may or may not be executed	loop is executed at least once

3.3.3 The `for` Statement

General Form:

```
for(initialization; test_condition; alteration)
{
    loop_body
}
```

For loop is an entry controlled loop. The execution of `for` statement is as follows:

- Initialization of the control variable is done first, using assignment statements such as `i = 1`. The variable `i` is known as loop-control variables
- The value of the control variable is tested using the test-condition. The `test_condition` is a relational expression, such as `i < 10` that determines when the loop will exit. If the condition is true, the body of the loop is executed; otherwise the loop is terminated.
- When the body of the loop is executed, the control is transferred back to the `for` statement. Now, the control variable is altered using statements such as `i = i + 1` and the new value of the control variable is again tested to see whether it satisfies the loop condition. If the condition is true, the body of the loop is executed again. This process continues until the `test_condition` fails.

An example is given below:

```
.....
for(i = 0; i <= 9; i++)
{
    printf("%d", x);
}
.....
```

This for loop is executed 10 times and prints digits 0 to 9 in one line.

Additional Features of `for` Statement

- More than one variable can be initialized at a time in the `for` statement. Multiple initializations should be separated by comma. For example

```
p = 1;
for(n = 0; n < 10; n++)
{
    ...
}
```

can be rewritten as

```
for(p = 1, n = 0; n < 10; n++)
{
    ...
}
```

- Alteration section can also have more than one part separated by comma. For example

```
for(n = 1, m = 50; n <= m; n = n + 1, m = m - 1)
{
    ...
}
```

- Test condition can be a compound relation involving logical operator. For example:

```
sum = 0;
for(i = 1; i < 20 && sum < 100; i++)
{
    sum = sum + i;
    printf("%d %d\n", i, sum);
}
```

- One or more sections of the `for` loop can be omitted. For example:

```
.....
m = 5;
for(; m != 100 ; )
{
    printf("%d\n", m);
    m = m + 5;
}
.....
```

In this `for` loop, both the initialization and alteration sections are omitted. The initialization has been done before the `for` loop and alteration of control variable is done inside the `for` loop. If the test condition is omitted, the loop becomes infinite and statements such as `break` need to be used inside the `for` loop to terminate the loop.

3.3.4 `break` Statement

The `break` statement is used to immediately exit from a loop. When a `break` is encountered inside a loop, the loop is terminated immediately and the program continues with the statement immediately following the loop. When the loops are nested, the `break` would only exit from the loop containing it. That is, `break` will exit only from a single loop. An example is given below:

```
for(i = 0; i < 10; i++)
{
    if(i == 5)
    {
        break;
    }
    printf("%d", i);
}
```

This `for` loop will print numbers from 0 to 4. When `i` gets the value 5, the `break` statement is executed and the loop is terminated.

3.3.5 `continue` Statement

The `continue` statement is used to skip a part of a loop. When `continue` statement is encountered inside a loop, the remaining statements inside the loop are skipped and the loop is continued with the next iteration. An example is given below:

```
for(i = 0; i < 10; i++)
{
    if(i == 5)
    {
        continue;
    }
    printf("%d", i);
}
```

This `for` loop will print numbers from 0 to 9 except 5. When `i` gets the value 5, the `continue` statement is executed and remaining statements inside the loop are skipped. The loop then continues with next iteration.

4. Arrays & Strings

4.1 Introduction

Array is a **fixed size** sequenced collection of elements of the **same data type** sharing a common name. For example, an array name `mark` can be used to represent a set of marks of a group of students. Individual marks can be referred by writing a number called **index** in square brackets after the array name. Arrays can be used not only for simple lists of values but also for tables of data in two or three or more dimensions.

4.2 One Dimensional Arrays

- A list of items can be a given one variable name. Such a variable is called a **one dimensional array**. Each item in the array can be accessed using an **index**.
- The index starts at 0. For example, to store a set of 5 numbers, a variable `number` can be declared as follows:

```
int number[5];
```

and the computer reserves 5 storage locations as follows:

	number[0]
	number[1]
	number[2]
	number[3]
	number[4]

Values to the array elements can be assigned as follows:

```
number[0] = 35;  
number[1] = 45;  
....
```

4.2.1 Declaration of One Dimensional Arrays

The general form of array declaration is:

```
type variable-name[size];
```

- type specifies the type of elements that will be contained in the array such as int, float or char.
- size specifies the maximum number of elements that can be stored inside the array. It **should be a numeric constant**.

For example:

```
float price[20];
```

declares the price to be an array containing 20 real elements with index (or subscripts) ranging from 0 to 19.

A complete example using a one dimensional array, that finds the sum of 10 numbers entered through the keyboard is given below.

```
void main()
{
    int i, a[10], total;
    total = 0;
    printf("Enter 10 numbers:");
    for(i = 0; i < 10; i++)
    {
        scanf("%d", &a[i])
    }
    for(i = 0; i < 10; i++)
    {
        total = total + a[i];
    }
    printf("Sum of the numbers = %d", sum);
}
```

4.2.2 Initialization of One Dimensional Arrays

An array can be initialized at either of the following stages:

1. At compile time
2. At run time

Compile Time Initialization

The general form of compile time initialization is:

```
type array_name[size] = {list_of_values};
```

Values in the list are separated by commas. For example, the statement:

```
int marks[3] = {10, 20, 30};
```

will declare an integer array of size 3 and will assign 10, 20, 30 as its elements. If the number of values in the list is less than the number of elements, then only that many elements will be initialized. The remaining elements will be set to zero automatically.

The size may be omitted. In such cases, the compiler will allocate enough space for all initialized elements. For example:

```
int marks[] = {10, 20, 30};
```

will declare the marks array to contain three elements with initial values 10, 20 and 30.

Run Time Initialization

Arrays can be initialized at run time using looping statements. For example:

```
.....
.....
for(i = 0; i < 100; i = i + 1)
{
    salary[i] = 0;
    scanf("%d", &mark[i]);
}
```

4.3 Two Dimensional Arrays

Two dimensional arrays can be used to store a table of data. For example, a matrix can be stored inside a two dimensional array. They can be considered as an array of one dimensional arrays. Two dimensional arrays can be declared as follows:

```
type array-name[row_size][column_size];
```

Each dimension of the arrays is indexed from zero to its maximum size minus one; the first index selects the row and the second index selects the column within that row. Two dimensional arrays are stored in memory as shown below.

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

4.3.1 Initialization of Two Dimensional Arrays**Compile Time Initialization**

Two dimensional arrays may be initialized by following their declaration with a list of initial values within braces. Initialization is done row by row. For example:

```
int a[2][3] = {1, 2, 3, 4, 5, 6};
```

initializes the elements of the first row to be 1, 2 and 3 respectively and elements of the second row to be 4, 5 and 6.

Run Time Initialization

Two dimensional arrays can be initialized at run time by using nested loops. For example:

```
for(i = 0; i < 10; i++)
{
    for(j = 0; j < 10; j++)
    {
        scanf("%d", &a[i][j]);
    }
}
```

4.4 Multidimensional Arrays

C allows three or more dimensional arrays. The general form of declaration of a multi dimensional array is:

```
type array-name[s1][s2][s3]...[sm];
```

where s_i is the size of the i^{th} dimension. Some examples are:

```
int survey[3][5][12];
float table[5][4][5][3];
```

Here, `survey` is a three dimensional array declared to contain 180 integer type elements. Similarly `table` is a four dimensional array containing 300 elements of floating point types.

The array `survey` may represent a survey data of rain fall during the last three years from January to December in five cities. A three dimensional array can be considered as an array of two dimensional arrays.

month	1	2		12	Year 1
city					
1					
.					
5					

month	1	2		12	Year 2
city					
1					
.					
5					

4.5 Strings

A string is a sequence (array) of characters that is treated as a single item. Any group of characters enclosed between a double quotation marks is a string constant.

4.5.1 Declaring and Initializing String Variables

C represents strings as character arrays. The general form of declaration of a string variable is:

```
char string_name[size];
```

where `size` determines the maximum number of characters in the `string_name`. For example:

```
char name[30];
char place[20];
```

- When the compiler assigns a character string to a character array, it automatically puts a **null character** (`'\0'`) at the end of the string. Therefore, `size` should be maximum number of characters in the string **plus one**.
- Strings can be initialized in either of the following forms:

```
char city[9] = "NEW YORK";
char city[9] = {'N','E','W',' ','Y','O','R','K','\0'};
```

Character array `city` has to be 9 elements long because, the string `"NEW YORK"` contains 8 characters and one element space is needed for null character `'\0'`. Also, if a character array is initialized by listing its elements, null character should be supplied explicitly.

- C permits to initialize a character array without specifying the size of the array. In such cases, the size of the array will be determined automatically, based on the number of elements initialized. For example, the statement:

```
char greeting[] = "Hello World";
```


defines the array `greeting` as 12 element array.

- Size can be declared larger than the string size in the initializer. For example:

```
char greeting[10] = "Hello";
```

In this case, the computer creates a character array of size 10, places the value `"Hello"` in it, terminates with null character and initialized all other elements to null. The storage will look like:

H	e	l	l	o	\0	\0	\0	\0	\0
---	---	---	---	---	----	----	----	----	----

- However size cannot be declared smaller than the string size. For example, the following declaration is invalid.

```
char greeting[3] = "Hello";
```

- Initialization cannot be separated from the declaration. For example, following statements will result in an error:

```
char greeting[10];
greeting = "Hello"; //Error
```

- A character array name cannot be used on the left hand side of an assignment operator. For example, the following statements will result in an error:

```
char greeting[10] = "Hello";
char greet[10];
greet = greeting; //Error
```

4.5.2 Reading Strings From Terminal

Using `scanf` function

An example for reading a string is given below:

```
char name[10];
scanf("%s", name);
```

- `scanf` terminates the input on the first space it finds. i.e if we enter `"NEW YORK"` as input, only `"NEW"` will be read into the array `name`.
- `scanf` function automatically terminates the string that is read, with a null character and therefore the character array should be large enough to hold the input string plus the null character.
- No `&` sign is required before the variable name while reading a string using `scanf` function
- In the above example if we give `"New"` as input string the name array is created in the memory as shown below:

N	e	w	\0	?	?	?	?	?	?
---	---	---	----	---	---	---	---	---	---

Unused locations will be filled with garbage.

Using `gets` Function

A string containing white spaces can be read from the keyboard using `gets` function. For example:

```
char line[50];
gets(line);
```

This will read a line of text from the keyboard.

4.5.3 Writing Strings to Function

Using `printf` function

The format `%s` can be used to display an array of characters that is terminated by the null character. For example:

```
printf("%s", name);
```

can be used to display the entire contents of the array, `name`.

Using `puts` function

A string can be printed using `puts` function. For example:

```
puts(line);
```

This will print the value of the string variable, `line` and then moves the cursor to the beginning of the next line on the screen.

4.5.4 Arithmetic Operations on Characters

Whenever a character constant or character variable is used in an expression, it is automatically converted into an integer value (ASCII Code) by the system. For example

```
x = 'a';
printf("%d", x);
```

will print the ASCII code of `a`, 97. It is also possible to perform arithmetic operations on the character constants and variables. For example,

```
x = 'z' - 1;
```

is a valid statement. In ASCII, value of `z` is 122 and therefore the statement will assign the value 121 to the variable `x`.

4.5.5 String Handling Functions

C does not support the comparison of two strings directly. That is, statements such as

```
if(name1 == name2)
if(name == "abc")
```

are not permitted. It is therefore needed to compare the two strings character by character. But the C library supports a large number of string handling functions that can be used to carry out many of the string manipulations. They are included in the `string.h` header file.

`strcat()` Function

- `strcat(string1, string2)` function joins two strings together. It takes the following form.

```
strcat(string1, string2);
```

`string1` and `string2` are character arrays. When the function `strcat()` is executed, the `string2` is appended to the `string1`.

- It does so by removing the null character at the end of the `string1` and placing `string2` there.
- The string at `string2` remains unchanged

Example:

Consider the following two strings:

part1 =	V	E	R	Y	\0								
part2 =	G	O	O	D	\0								

Execution of the statement

```
strcat(part1, part2);
```

will result in:

part1 =	V	E	R	Y		G	O	O	D	\0							
part2 =	G	O	O	D	\0												

- We must make sure that the size of `string1` is large enough to hold the final string.

`strcmp(string1, string2)` Function

- The `strcmp()` function compares two strings
- This function returns 0 if the compared strings are equal. If they are not, it returns the numeric difference between the first non matching characters in the strings
- It takes the following form

```
strcmp(string1, string2);
```

`string1` and `string2` may be string variables or string constants. Some examples are

```
strcmp(name1, name2);
strcmp(name1, "John");
strcmp("apple", "cat");
```

- If the numeric difference is a positive value, `string1` is alphabetically above `string2`. If the numeric difference is a negative value, `string1` is alphabetically below `string2`. For example

```
strcmp("their", "there");
```

will return a value of -9 which is the numeric difference between the ASCII values of "i" and "r".

`strcpy(string1, string2)` Function

- This function works like a string assignment operator
- It takes the following form

```
strcpy(string1, string2);
```

will assign the content of `string2` to `string1`. `string2` may be a string constant or string variable. For example

```
strcpy(city, "bangalore");
```

will assign the string "bangalore" to "city".

`strlen(string1)` Function

- This function returns the number of characters (excluding null characters) in a string. It takes the following form

```
n = strlen(string1);
```

where `n` is an integer variable. It will get the length of the string variable `string1`.

```
char name[30] = "Hello";
printf("String length: %s", strlen(name));
```

will print out 5.

`strrev(string1)` Function

- This function reverses a given string. It takes the following form.

```
strrev(string1);
```

For example:

```
char name[30] = "Hello";
printf("String after reversing: %s", strrev(name));
```

will print out olleH.

strncpy(s1,s2,n) Function

- strncpy() copies the left most n characters of the source string to the target string variable.
- It takes the following form:

```
strncpy(s1,s2,n);
```

where n may be an integer variable or integer constant. This will copy the first n characters of s2 to s1.

- Since the first n characters may not contain the null character, it has to be placed explicitly at the $n + 1^{th}$ position.

strncmp(s1,s2,n) Function

- This function takes the following form

```
strncmp(s1,s2,n);
```

- This compares the left most n characters of s1 to s2 and returns
 - 0 if they are equal
 - negative number, if s1 substring is less than s2 and
 - positive number otherwise

strncat(s1,s2,n) Function

- This function takes the following form

```
strncat(s1,s2,n);
```

- This will concatenate the left most n characters of s2 to the end of s1.

strstr() Function

- This function takes the following form

```
strstr(s1,s2);
```

The function searches the string s1 to see whether the string s2 is contained in s1. If yes, the function returns the position of the first occurrence of the substring. Otherwise it returns a null.

4.5.6 Table of Strings

A list of words can be treated as a table of strings. A two dimensional character array can be used to store the entire list. For example, a character array student[30][15] can be used to store a list of 30 names each of length not more than 15 characters.

Consider a list of 5 cities. This can be stored in an array as shown below

```
char city[][]
{
    "cochin",
    "calicut",
    "kannur",
    "trichur",
    "kasaragod"
};
```

To access the name of the i^{th} city in the list, we write

```
city[i-1]
```

Therefore city[0] denotes "cochin", city[1] denotes "calicut" and so on. Therefore, once an array is declared as a two dimensional array, it can be used like a one dimensional array in further manipulations. That is, the table can be treated like a list of strings

Bibliography

Books

- [1] E. Balagurusamy. *Programming in ANSI C*. McGraw Hill Education (India), 2017. ISBN: 9789339219666. URL: <https://books.google.co.in/books?id=b2b5swEACAAJ>.
- [2] B.W. Kernighan and D.M. Ritchie. *The C Programming Language*. Prentice-Hall software series. Prentice Hall, 1988. ISBN: 9780131103627. URL: <https://books.google.co.in/books?id=161QAAAAAAAJ>.
- [3] I.T.L.E.S. Limited and I. Esl. *Introduction to Computer Science*. Pearson Education, 2004. ISBN: 9788131704363.